

Practical No : 5

Aim: To implement SVM

code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# ---- Load Titanic dataset (replace with your df if already loaded) ----
titanic = df

# Select features (Age, Fare) and target (Survived)
df = titanic[['Age', 'Fare', 'Survived']].dropna()

X = df[['Age', 'Fare']].values
y = df['Survived'].values

# ---- Split data ----
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# ---- Train SVM ----
svm_model = SVC(kernel='linear', C=1)
svm_model.fit(X_train, y_train)

# ---- Predictions ----
y_pred = svm_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# ---- Confusion Matrix ----
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Died", "Survived"], yticklabels=["Died", "Survived"])
plt.title("Confusion Matrix")
plt.show()

# ---- Decision Boundary Function ----
def plot_decision_boundary(model, X, y):
    h = 0.2
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 10, X[:, 1].max() + 10

    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.3)
    sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=y, palette="coolwarm", edgecolor='k')
    plt.title("SVM Decision Boundary (Age vs Fare)")
    plt.xlabel("Age")
    plt.ylabel("Fare")
    plt.show()

plot_decision_boundary(svm_model, X, y)

# ---- Support Vectors Visualization ----
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X[:, 0], y=X[:, 1], hue=y, palette="coolwarm", s=60, edgecolor='k')
plt.scatter(svm_model.support_vectors_[0], svm_model.support_vectors_[1], s=150, facecolors='none', edgecolors='black', linewidths=1.5, label='Support Vectors')
plt.title("Support Vectors (Age vs Fare)")
plt.xlabel("Age")
plt.ylabel("Fare")
plt.legend()
plt.show()
```

Accuracy: 0.7229437229437229

Classification Report:

	precision	recall	f1-score	support
0	0.74	0.95	0.83	168
1	0.47	0.13	0.20	63
accuracy			0.72	231
macro avg	0.61	0.54	0.52	231
weighted avg	0.67	0.72	0.66	231

